

halfedge：多边形三角化模块设计文档

src/triangulation.rs

halfedge 库

2026-07-04

目录

1	模块定位	1
2	扇形三角化	2
2.1	算法	2
2.2	前置条件	2
2.3	退化处理	2
2.4	复杂度	2
3	耳剪裁三角化	2
3.1	基本概念	2
3.2	算法	2
3.3	安全兜底	3
3.4	3D 投影	3
3.5	流程图	4
4	辅助函数（私有）	4
5	点在三角形内：面积分解	4
6	退化与错误处理	4
7	复杂度	5
8	测试覆盖	5
9	设计取舍	5

1 模块定位

triangulation.rs 提供 2D/3D 多边形三角化算法，返回三角面顶点索引 `Vec<[usize; 3]>`。不依赖 `MeshStorage`，是纯几何工具，供 `io.rs`（n-gon 解析）与未来可能的高级构建器复用。

算法	API	适用范围
扇形三角化	<code>fan_triangulation(&[[f64;2]])</code>	2D 凸多边形 (CCW)
	<code>fan_triangulation_3d(&[[f64;3]])</code>	3D 共面凸多边形
耳剪裁	<code>ear_clipping(&[[f64;2]])</code>	2D 简单多边形 (凹允许)
	<code>ear_clipping_3d(&[[f64;3]])</code>	3D 共面简单多边形

2 扇形三角化

2.1 算法

对 n 顶点多边形 $v_0, v_1, \dots, v_{n-1}$, 从 v_0 出发扇形展开:

$$T_i = (v_0, v_{i-1}, v_i), \quad i = 2, 3, \dots, n-1.$$

共生成 $n-2$ 个三角形。索引形式: $[0, i-1, i]$ 。

2.2 前置条件

- 凸多边形 (凹多边形会生成位于外部的三角形);
- CCW 朝向;
- 3D 版本要求所有顶点共面 (不验证, 调用方负责)。

2.3 退化处理

$n < 3$ 返回空 `Vec`; $n = 3$ 自然返回单三角形 $[0, 1, 2]$ 。

2.4 复杂度

$O(n)$, 单次线性扫描, `Vec::with_capacity(n - 2)` 预分配。

3 耳剪裁三角化

3.1 基本概念

耳 (ear): 多边形的一个顶点 v_i , 其相邻两顶点 v_{i-1}, v_{i+1} 形成的三角形 (v_{i-1}, v_i, v_{i+1}) 完全位于多边形内部, 且不包含其他顶点。

定理 (Meisters 二耳定理): 任何 $n \geq 4$ 顶点的简单多边形至少有 2 个不重叠的耳。故可迭代摘除耳直到只剩三角形。

3.2 算法

1. 若输入 CW, 反向为 CCW;
2. 维护 `indices = [0, 1, ..., n-1]`;

3. 当 `indices.len() > 3`:

(a) 对每个 i , 邻居 $prev = i - 1$, $next = i + 1$:

(b) **凸性测试**: $cross = (b - a) \times (c - a)$, $cross \leq 0$ (反射或共线) 跳过;

(c) **包含测试**: 检查所有其他顶点 $j \notin \{prev, i, next\}$ 是否在三角形 (a, b, c) 内 (面积分解法);

(d) 若通过两测试, 输出 `[indices[prev], indices[i], indices[next]]`, 从 `indices` 移除 i , **break** 重启扫描;

4. 若一轮未找到耳 (`!found_ear`), **break** 防止死循环;

5. 若剩余 3 顶点, 输出最后一个三角形。

3.3 安全兜底

`iter_count > indices.len()2` 时强制 **break**, 对自相交多边形防止死循环。返回已收集的三角形 (部分结果)。

3.4 3D 投影

`ear_clipping_3d`:

1. Newell 方法计算多边形法向 $\vec{n}$:

$$n_x = \sum_i (y_i - y_{i+1})(z_i + z_{i+1}) \quad (1)$$

$$n_y = \sum_i (z_i - z_{i+1})(x_i + x_{i+1}) \quad (2)$$

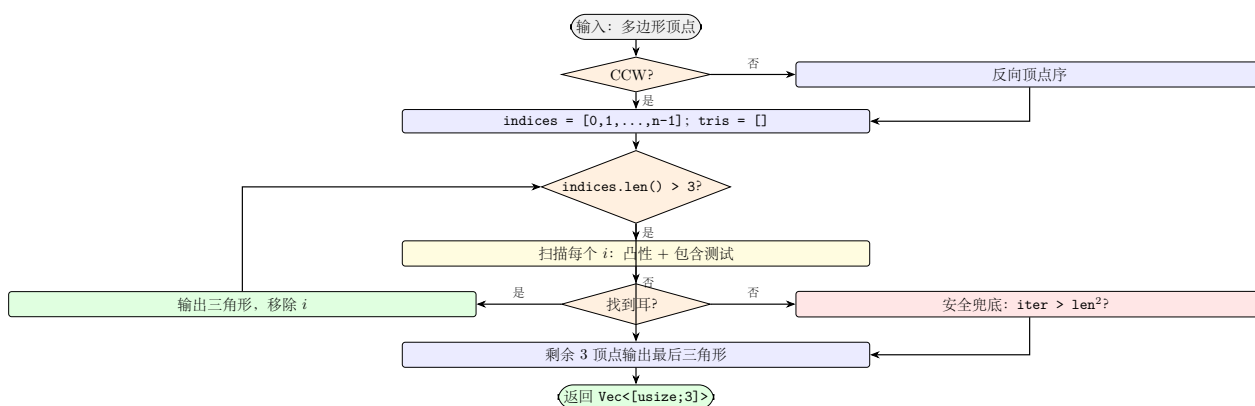
$$n_z = \sum_i (x_i - x_{i+1})(y_i + y_{i+1}) \quad (3)$$

2. 选择 $|n_k|$ 最大的轴 k 投影到另外两轴:

- $k = 0$ (drop x) $\rightarrow$ 用 (y, z)
- $k = 1$ (drop y) $\rightarrow$ 用 (x, z)
- $k = 2$ (drop z) $\rightarrow$ 用 (x, y)

3. 调用 2D `ear_clipping`, 返回的索引直接对应原 3D 顶点。

3.5 流程图



4 辅助函数（私有）

函数	语义
<code>v2_sub(a, b)</code>	二维向量减法
<code>v2_cross(a, b)</code>	二维标量叉积 $a_x b_y - a_y b_x$
<code>signed_area(a, b, c)</code>	三角形有符号面积（CCW 为正）
<code>point_in_triangle(p, a, b, c)</code>	面积分解法判定点在三角形内
<code>is_ccw(poly)</code>	Shoelace 公式判定多边形朝向

5 点在三角形内：面积分解

点 p 在三角形 (a, b, c) 内当且仅当三个子三角形面积之和等于原三角形面积：

$$|A(p, b, c)| + |A(a, p, c)| + |A(a, b, p)| \approx |A(a, b, c)|.$$

容差 $10^{-12} \cdot \max(|A|, 10^{-12})$ 防止退化三角形除零。

6 退化与错误处理

所有函数无 `Result`、无 `panic`。退化输入静默返回部分结果：

输入	返回
$n < 3$ （任意函数）	空 <code>Vec</code>
$n = 3$ （耳剪裁）	<code>vec![[0, 1, 2]]</code>
CW 输入（耳剪裁）	静默反向后三角化
自相交多边形	部分三角形列表（提前 <code>break</code> ）
迭代超限	部分三角形 + 最后三角形（若剩余 3 顶点）

调用方需自行验证输入几何以获得正确结果。

7 复杂度

函数	时间复杂度
<code>fan_triangulation</code>	$O(n)$
<code>fan_triangulation_3d</code>	$O(n)$
<code>ear_clipping</code>	$O(n^2)$ (每耳摘除扫描 $O(m)$ 剩余顶点)
<code>ear_clipping_3d</code>	$O(n)$ (投影) + $O(n^2)$ (耳剪裁) = $O(n^2)$

8 测试覆盖

测试名	验证点
<code>fan_triangulation_quad</code>	单位正方形 (CCW) $\rightarrow$ 2 三角形 $[0, 1, 2], [0, 2, 3]$
<code>fan_triangulation_empty</code>	空输入 / 单顶点 $\rightarrow$ 0 三角形
<code>ear_clipping_convex_quad</code>	凸四边形耳剪裁 $\rightarrow$ 2 三角形
<code>ear_clipping_concave_l_shape</code>	L 形凹六边形 $\rightarrow$ 4 三角形 ($n - 2 = 4$)
<code>ear_clipping_3d_square</code>	$z = 0$ 平面正方形 3D 耳剪裁 $\rightarrow$ 2 三角形
<code>ear_clipping_triangle</code>	3 顶点多边形 $\rightarrow$ 1 三角形

全模块共 6 个单元测试，全库共 371 个。

9 设计取舍

- **为何保留扇形三角化？** 凸多边形场景下 $O(n)$ 远优于耳剪裁 $O(n^2)$ ，且实现简单。OBJ n-gon 解析默认凸性假设时可使用。
- **为何耳剪裁用 $O(n^2)$ 而非 $O(n \log n)$ 改进版？** 经典实现足够清晰，对典型 n-gon ($n \leq 8$) 性能差异可忽略。改进版需维护单调剖分与桥边检测，实现复杂度大幅上升。
- **为何不返回 Result？** 三角化对任意输入都有「最大努力」结果，即使自相交也能返回部分三角形。让调用方决定是否接受部分结果。
- **为何 3D 投影选最大法向分量轴？** 投影到该轴对应的「最平行」平面，面积失真最小，避免退化（如投影到与多边形垂直的平面会让所有点共线）。